КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМ. ІГОРЯ СІКОРСЬКОГО

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №2

на тему:

«Реалізація алгоритмів генерації ключів гібридних криптосистем»

Виконали:
студентки групи ФІ-62 мн
Дегтярьова Марія
Тимошенко Марія

Київ – 2026

Зміст

1 Дослідження принципів генерації криптографічних ключів 2

2 Дослідження генерації простих чисел 5

3 Практичне дослідження генерації випадкових значень 8

4 Дослідження генерації асиметричних ключів 10

5 Аналіз ситуацій порушення безпеки 12

6 Життєвий цикл криптографічного ключа 14

Висновки 14

Перелік використаних джерел 15

Мета роботи: дослідження принципів формування криптографічно стійких випадкових послідовностей, генерації ключового матеріалу, тестування та генерації простих чисел для асиметричних криптосистем, а також основних вимог до життєвого циклу криптографічних ключів.

1 Дослідження принципів генерації криптографічних ключів

1.1 Що таке ентропія у контексті генерації криптографічних ключів? Які джерела ентропії можуть використовувати сучасні програмно-апаратні системи?

Ентропія у контексті криптографії характеризує ступінь невизначеності випадкових даних, які використовуються для створення криптографічних ключів. Чим вища ентропія, тим складніше зловмиснику передбачити значення ключа або підібрати його перебором. Навіть якщо ключ має достатню довжину, його безпека може бути низькою, якщо він згенерований на основі передбачуваних даних.

Сучасні програмно-апаратні системи можуть використовувати різні джерела ентропії. До них належать фізичні процеси, зокрема тепловий шум електронних компонентів і нестабільність частоти коливань електронних схем. Також можуть враховуватися часові характеристики системних подій, рухи миші, натискання клавіш та інші непередбачувані події. Операційні системи можуть поєднувати дані з кількох джерел, обробляти їх і використовувати для ініціалізації криптографічно стійких генераторів випадкових чисел. При цьому важливо не лише збирати випадкові дані, а й оцінювати їхню якість та рівень ентропії [1, 3].

1.2 У чому полягає різниця між TRNG, PRNG та криптографічно стійким генератором псевдовипадкових чисел (CSPRNG)?

TRNG (True Random Number Generator) є генератором справжніх випадкових чисел, який використовує фізичні процеси для отримання випадкових значень. Такі генератори можуть спиратися на тепловий шум або інші фізичні явища, результати яких важко передбачити. Їхньою особливістю є те, що випадковість формується на основі фізичного джерела, а не лише обчислень.

PRNG (Pseudorandom Number Generator) є генератором псевдовипадкових чисел, який за певним алгоритмом формує послідовність значень на основі початкового стану. Такі послідовності можуть мати хороші статистичні властивості, однак за однакових початкових умов генератор відтворює ті самі значення.

CSPRNG (Cryptographically Secure Pseudorandom Number Generator) є криптографічно стійким генератором псевдовипадкових чисел. Він також використовує алгоритм

для формування послідовності, але має додаткову вимогу щодо непередбачуваності результатів. Знання попередніх значень не повинно давати змоги ефективно визначити наступні значення або відновити внутрішній стан генератора за умови дотримання вимог до його реалізації та ініціалізації. CSPRNG зазвичай використовує достатньо випадкове початкове значення, отримане з надійного джерела ентропії. Саме такі генератори застосовують для створення криптографічних ключів та інших секретних даних [1, 2].

1.3 Яку роль виконує початкове значення (seed) генератора псевдовипадкових чисел? Які наслідки може мати можливість передбачити або відтворити seed?

Початкове значення (seed) використовується для ініціалізації генератора псевдовипадкових чисел. На його основі формується початковий стан, з якого алгоритм надалі генерує послідовність значень. Для звичайного детермінованого генератора однакове початкове значення за однакових умов призводить до відтворення тієї самої послідовності. Це є корисним під час моделювання та проведення експериментів, оскільки дозволяє повторити отримані результати.

У криптографії можливість передбачити або відтворити seed становить серйозну загрозу безпеці. Якщо зловмисник знає початкове значення або може перебрати невелику кількість можливих варіантів, він може відтворити послідовність генератора та визначити створені на її основі ключі. У такому випадку навіть ключ великої довжини може виявитися вразливим, оскільки фактична кількість можливих ключів обмежується невизначеністю початкового значення. Тому для криптографічних генераторів важливо використовувати достатньо випадковий seed, захищати внутрішній стан і правильно реалізовувати механізми ініціалізації та оновлення стану [1, 2].

1.4 Чому звичайні генератори псевдовипадкових чисел, призначені для моделювання, ігор або статистичних задач, не повинні використовуватися для генерації криптографічних ключів?

Звичайні генератори псевдовипадкових чисел призначені насамперед для отримання послідовностей, які мають необхідні статистичні властивості та підходять для моделювання, ігор або статистичних експериментів. Проте такі генератори не обов'язково забезпечують захист від спроб передбачити наступні значення або відновити їх за відомими результатами. Тому статистично випадкова на вигляд послідовність не обов'язково є безпечною для використання в криптографії.

Якщо криптографічний ключ створено за допомогою звичайного генератора, зловмисник може скористатися особливостями алгоритму, передбачуваним початковим значенням або відомими результатами його роботи, щоб звузити множину можливих ключів. Це може призвести до розкриття зашифрованих даних або компрометації системи автен-

тифікації. Наприклад, у мові Python модуль `random` призначений для моделювання та статистичних задач, тоді як модуль `secrets` надає засоби для отримання криптографічно стійких випадкових значень. Отже, для генерації криптографічних ключів потрібно використовувати спеціально призначені криптографічно стійкі генератори, які спираються на належне джерело випадковості та відповідають вимогам безпеки [1, 4, 5].

1.5 Яким чином сучасні операційні системи надають програмам криптографічно стійкі випадкові значення? Навести приклад відповідного API для однієї операційної системи або мови програмування.

Сучасні операційні системи генерують криптографічно стійкі випадкові значення за допомогою збирання фонової ентропії з апаратних подій, переривань від пристроїв введення, таймерів та мережевої активності, обробляючи ці дані всередині ядерної підсистеми відповідно до стандартів серії NIST SP 800-90 [11]. Ядро ОС акумулює цей шум у спеціальному пулі та за допомогою криптографічних алгоритмів надає прикладним програмам непередбачувані байти за запитом. Наприклад, у мові програмування Python стандартна функція `os.urandom(n)` звертається безпосередньо до захищених системних інтерфейсів операційної системи (таких як криптографічні функції `BCryptGenRandom` у Windows або системний виклик `getrandom()` у Linux), забезпечуючи надійне отримання випадкових послідовностей для криптографічних потреб [12].

1.6 Чим принципово відрізняється генерація випадкового симетричного ключа від генерації пари ключів асиметричної криптосистеми?

Принципова відмінність полягає в алгоритмічній складності та математичній структурі одержуваного ключового матеріалу. Генерація симетричного ключа зводиться до виклику криптографічно стійкого генератора випадкових чисел (CSPRNG) для отримання масиву випадкових байтів фіксованої довжини, наприклад, 256 біт для алгоритму AES-256 [12]. Натомість генерація пар асиметричних ключів, таких як відкритий та закритий ключі для RSA чи протоколів на еліптичних кривих, є складним обчислювальним процесом, який вимагає генерації великих випадкових простих чисел за допомогою імовірнісних тестів простоти, перевірки їхніх взаємних математичних властивостей та обчислення математичних зв'язків між ключами за модулем відповідно до специфікацій стандарту PKCS #1 [13].

1.7 Чи гарантує використання алгоритму AES-256 рівень захисту у 256 біт, якщо ключ отримано з недостатньо випадкового або передбачуваного джерела? Обґрунтувати відповідь.

Використання алгоритму AES-256 не гарантує заявленого рівня захисту, якщо ключ було згенеровано з передбачуваного або низькоентропійного джерела [14]. Сам криптографічний алгоритм є математично стійким, проте практична стійкість усієї системи обмежується початковою ентропією самого ключа. Якщо зловмисник зможе виявити закономірність генерації або підібрати початкове значення (seed) за допомогою методу повного перебору (brute-force), він зможе відтворити ключовий матеріал і розшифрувати дані без необхідності здійснювати складні математичні атаки на сам симетричний шифр [14].

1.8 Які основні етапи життєвого циклу криптографічного ключа необхідно враховувати, крім безпосередньої генерації ключа?

Окрім етапу генерації, життєвий цикл криптографічного ключа охоплює: безпечне зберігання та використання в ізольованому середовищі, планову або екстрену ротацію (заміну) для мінімізації ризиків компрометації обсягів даних, створення захищених резервних копій за необхідності, операційне відкликання у разі виявлення загроз, а також гарантоване та безповоротне знищення (*zeroization*) ключів після завершення терміну їх експлуатації [14, 15].

2 Дослідження генерації простих чисел

2.1 Для чого при генерації ключів RSA необхідні великі прості числа?

У криптосистемі RSA великі прості числа використовуються для побудови приватного та публічного ключів. Спочатку генеруються два різних великих простих числа p і q, після чого обчислюється їхній добуток $n = pq$. Число n є частиною приватного ключа, а його факторизація, тобто знаходження початкових множників p і q, за достатньо великої довжини чисел є обчислювально складною задачею.

Знання цих простих множників дозволяє обчислити значення, необхідне для побудови публічного ключа. Тому безпека RSA значною мірою залежить від складності факторизації числа n. Використання великих простих чисел ускладнює таку атаку та забезпечує необхідний рівень криптографічної стійкості. При цьому важливо, щоб числа були різними, достатньо великими та згенерованими надійним способом [6, 7].

2.2 Чому на практиці для великих чисел використовуються ймовірнісні тести простоти?

Для перевірки простоти великих чисел на практиці часто використовують ймовірнісні тести, оскільки вони дозволяють ефективно перевіряти числа, що містять сотні або тисячі бітів. Точна перевірка простоти простими методами може потребувати значних обчислювальних ресурсів, особливо коли йдеться про числа, які використовуються в криптографії.

Ймовірнісні тести перевіряють число за певними математичними властивостями та виконують обчислення для одного або кількох обраних значень. Якщо тест виявляє порушення необхідної властивості, число є складеним. Якщо ж усі перевірки пройдено, число вважається ймовірно простим, оскільки залишається певна ймовірність помилкового результату. Виконання кількох незалежних раундів тестування дозволяє значно зменшити ймовірність такої помилки. Завдяки швидкості та практичній надійності ці методи широко застосовуються під час генерації ключів RSA [9, 10].

2.3 У чому полягає основна ідея тесту Ферма та чому позитивний результат цього тесту не гарантує простоту числа?

Тест Ферма ґрунтується на малій теоремі Ферма, відповідно до якої для простого числа n і цілого числа a, що не ділиться з n на спільний дільник, виконується співвідношення $a^{n-1} \equiv 1 \pmod{n}$. На основі цієї властивості можна перевіряти, чи відповідає число необхідній умові простоти. Якщо рівність не виконується для вибраного значення a, число гарантовано є складеним.

Однак виконання цієї умови не гарантує, що число просте. Існують складені числа, які задовольняють умову тесту для деяких або навіть для всіх допустимих значень основи. Наприклад, числа Кармайкла проходять тест Ферма для всіх основ, взаємно простих із цими числами. Тому складене число може бути помилково прийняте за ймовірно просте. Через це тест Ферма самостійно не забезпечує достатньої надійності для перевірки простоти чисел, що використовуються під час генерації криптографічних ключів [8].

2.4 Яку проблему тесту Ферма частково вирішують більш досконалі ймовірнісні тести?

Більш досконалі ймовірнісні тести простоти, зокрема тест Міллера — Рабіна, дозволяють виявляти частину складених чисел, які можуть пройти тест Ферма. Для цього використовуються додаткові математичні властивості числа, пов'язані з розкладом числа $n - 1$ на степінь двійки та непарний множник. Перевіряються не лише результати піднесення до степеня за модулем, а й проміжні значення, які повинні задовольняти певні умови.

Завдяки цьому тест Міллера — Рабіна виявляє більше складених чисел і має кон-

трольовану ймовірність помилкового прийняття складеного числа за ймовірно просте. Зі збільшенням кількості раундів ця ймовірність зменшується. Тому на практиці тест Міллера — Рабіна є значно надійнішим за простий тест Ферма та широко використовується для перевірки кандидатів у прості числа під час генерації ключів RSA [9, 10].

2.5 Який принцип роботи тесту Міллера–Рабіна та яке значення має кількість його ітерацій?

Тест Міллера–Рабіна є високоефективним імовірнісним поліноміальним тестом простоти, який базується на перевірці властивостей малих теорем Ферма та властивостей квадратних коренів з одиниці за модулем досліджуваного числа [16]. Число $n-1$ подається у вигляді добутку степеня двійки та непарного числа ($n-1=2^s\cdot d$), після чого для випадкових основ виконується серія модульних піднесеннь до степеня. Кількість ітерацій k безпосередньо визначає надійність тесту: з кожною новою ітерацією верхня межа ймовірності того, що складене число помилково визнано простим, зменшується вдвічі (стає меншою за 4^{-k}) [16]. Для практичних криптографічних потреб обирають достатню кількість ітерацій, що гарантує мізерну ймовірність помилки, яка повністю відповідає стандартам безпеки [16, 15].

2.6 Чому дуже мала ненульова ймовірність помилки тесту простоти може бути прийнятною при практичній генерації криптографічних ключів?

Дуже мала ненульова ймовірність помилки тесту простоти є цілком прийнятною на практиці, оскільки вона є значно меншою за ймовірність виникнення апаратних збоїв чи помилок пам'яті комп'ютера під час виконання обчислень через зовнішні чинники, такі як космічне випромінювання [16, 15]. Якщо ймовірність хибнопозитивного результату тесту становить менше ніж 2^{-80} або 2^{-128}, то ризик випадково обрати складене число замість простого настільки незначний, що він не має жодного реального впливу на загальну стійкість асиметричної криптосистеми [16].

2.7 Які вимоги, крім достатньої довжини, висуваються до простих чисел, що використовуються при формуванні RSA-ключів?

Окрім достатньої бітової довжини (від 1024 до 4096 біт), прості числа повинні задовольняти низку суворих криптографічних вимог, які гарантують стійкість модуля проти атак факторизації [16, 17]. Зокрема, числа мають бути випадковими та непередбачуваними, їхня різниця не повинна бути занадто малою (інакше модуль можна легко розкласти методом Ферма), а порядок мультиплікативної групи повинен мати великі прості дільники для захисту від спеціалізованих атак (наприклад, алгоритму Поларда) [16, 17].

2.8 Порівняння тестів Ферма, Соловея–Штрассена та Міллера–Рабіна:

Тест Ферма є найпростішим у реалізації, проте він має істотний недолік у вигляді існування чисел Кармайкла, на яких тест завжди дає хибний позитивний результат для складених чисел, що робить його непридатним для надійної криптографії [16]. Тест Соловея–Штрассена базується на використанні символу Якобі та критерію Ейлера, він не має таких слабких місць, як числа Кармайкла, але є складнішим в обчислювальній реалізації [16, 13]. Тест Міллера–Рабіна є найдосконалішим, найшвидшим і є стандартом де факто у сучасних криптографічних бібліотеках і нормативних документах (наприклад, утилітах OpenSSL та специфікаціях ANSI X9.80 / FIPS), оскільки він забезпечує максимальну надійність виявлення складених чисел за мінімальної кількості обчислювальних кроків [16, 15, 13].

3 Практичне дослідження генерації випадкових значень

3.1 Використання звичайного генератора псевдовипадкових чисел

Для дослідження відтворюваності псевдовипадкових чисел було використано стандартний модуль `random` мови програмування Python. Спочатку було встановлено фіксоване початкове значення `seed = 42`, після чого згенеровано послідовність із десяти випадкових цілих чисел у діапазоні від 0 до 255. Далі генерацію було повторено з тим самим початковим значенням, а отримані послідовності порівняно між собою.

```
import random

seed = 42
n = 10

random.seed(seed)
sequence1 = [random.randint(0, 255) for _ in range(n)]
random.seed(seed)
sequence2 = [random.randint(0, 255) for _ in range(n)]

print("I try:", sequence1)
print("II try:", sequence2)
print("Same?", sequence1 == sequence2)
```

У результаті експерименту було встановлено, що обидві послідовності повністю збігаються.

```
I try: [57, 12, 140, 125, 114, 71, 52, 44, 216, 16]
II try: [57, 12, 140, 125, 114, 71, 52, 44, 216, 16]
Same? True
```

Це пояснюється тим, що генератор `random` є детермінованим і за однакового початкового значення та однакових умов генерує ту саму послідовність чисел.

Якби отримані значення використовувалися для формування криптографічних ключів, така відтворюваність могла б становити загрозу безпеці. Якщо зловмисник знає початкове значення або може його вгадати, він може повторити роботу генератора та отримати ті самі значення, що використовувалися під час створення ключа. Це дозволило б звузити пошук можливих ключів і потенційно призвести до розкриття захищених даних. Тому звичайний генератор `random`, призначений для моделювання та статистичних задач, не слід використовувати для генерації криптографічних ключів. Для цього необхідно застосовувати криптографічно стійкі генератори випадкових чисел.

3.2 Використання криптографічно стійкого генератора

Для практичної демонстрації роботи криптографічно стійкого генератора випадкових чисел (CSPRNG) використано стандартну бібліотеку `cryptography` у мові Python [12, 11].

Імпорт необхідних модулів:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

3.2.1 Генерація криптографічних ключів

```
key1 = AESGCM.generate_key(bit_length=256)
key2 = AESGCM.generate_key(bit_length=256)

print("Key 1 (hex):", key1.hex())
print("Key 2 (hex):", key2.hex())
```

Результат запуску:

```
Key 1 (hex): a1e668462ba47835e8f5d20e6753997a9651ad0c05fd492fe37e47b671c90583
Key 2 (hex): 75d8e1eb2917f2c811df249e01661114570551273bb1851d63f7947aaa3115f1
```

3.2.2 Повторний запуск програми

Під час повторного запуску програми (повторного виконання скрипту) отримано такі результати:

```
Key 1 (hex): d7d04a1ff68c38309478f8221868df67fa601470edbc2950b176611de874ce25
Key 2 (hex): 85fb6c2041ab72c0c2dcaaf5b4fb05e64712585629c7b864a02755fad8d70312
```

3.2.3 Порівняння результатів з експериментом 3.1

У звичайному некриптографічному генераторі (розглянутому в п. 3.1) за встановленого фіксованого початкового значення (*seed*) згенеровані послідовності повністю від-

творюються при кожному перезапуску програми. Натомість при використанні CSPRNG ключі щоразу є повністю унікальними та непередбачуваними, а їх відтворення чи повторення після перезапуску є неможливим.

3.2.4 Джерело криптографічно стійких випадкових значень

Програмний інтерфейс отримує випадкові значення безпосередньо з операційної системи, наприклад, через криптографічні функції `BCryptGenRandom` у Windows або системний виклик `getrandom()` у Linux [11, 12]. Ядро ОС динамічно акумулює фоновий апаратний шум (ентропію) від переривань пристроїв введення, таймерів і системних подій, гарантуючи високу якість та непередбачуваність ключового матеріалу.

4 Дослідження генерації асиметричних ключів

4.1 Генерація RSA-ключа довжиною 2048 біт

Для генерації RSA-ключа довжиною 2048 біт використано криптографічну бібліотеку `cryptography`. Час генерації вимірювався за допомогою функції `time.perf_counter()`. Експеримент було проведено тричі.

```
from cryptography.hazmat.primitives.asymmetric import rsa
import time

for i in range(1, 4):
    start = time.perf_counter()

    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    end = time.perf_counter()
    print(f"Try (2048) {i}: {end - start:.4f} c")
```

Під час трьох запусків час генерації RSA-ключа довжиною 2048 біт становив 0,0289 с, 0,0950 с і 0,0439 с відповідно. Середній час генерації дорівнював приблизно 0,0559 с.

4.2 Генерація RSA-ключа довжиною 4096 біт

Для дослідження впливу довжини ключа на час генерації експеримент було повторено для RSA-ключа довжиною 4096 біт.

```
from cryptography.hazmat.primitives.asymmetric import rsa
import time

```

```
for i in range(1, 4):
    start = time.perf_counter()

    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=4096
    )

    end = time.perf_counter()
    print(f"Try (4096) {i}: {end - start:.4f} c")
```

Час генерації RSA-ключа довжиною 4096 біт становив 1,2870 с, 0,2607 с і 0,3652 с відповідно. Середній час генерації дорівнював приблизно 0,6376 с. Результати демонструють помітні відмінності між окремими запусками, що можуть бути пов'язані з особливостями пошуку простих чисел та навантаженням на систему.

4.3 Порівняння часу генерації

У таблиці наведено результати вимірювання часу генерації RSA-ключів різної довжини.

Табл. 1: Результати генерації RSA-ключів

Довжина ключа, біт	**Номер запуску**	**Час генерації, с**
2048	1	0,0289
2048	2	0,0950
2048	3	0,0439
4096	1	1,2870
4096	2	0,2607
4096	3	0,3652

Середній час генерації ключа довжиною 2048 біт становив приблизно 0,0559 с, а ключа довжиною 4096 біт — 0,6376 с. Таким чином, у проведеному експерименті генерація ключа довжиною 4096 біт у середньому тривала приблизно в 11,4 раза довше.

4.4 Аналіз результатів

Результати експерименту показують, що збільшення довжини RSA-ключа з 2048 до 4096 біт суттєво впливає на час генерації. Це пов'язано зі збільшенням розміру чисел, з якими виконуються арифметичні операції, та складністю пошуку відповідних простих чисел.

Час генерації може відрізнятися навіть для ключів однакової довжини. Під час кожного запуску використовуються нові випадкові значення, тому кількість спроб, необхідних для знаходження простих чисел, може бути різною. Крім того, на результати

вимірювань впливають навантаження на процесор та інші процеси, що виконуються на комп'ютері.

Важливу роль у генерації RSA-ключів відіграє криптографічно стійкий генератор псевдовипадкових чисел (CSPRNG), який забезпечує непередбачуваність значень, що використовуються для формування кандидатів на прості числа. Ці кандидати проходять тести простоти, після чого знайдені прості числа використовуються для побудови ключової пари.

Збільшення довжини RSA-ключа також впливає на швидкість криптографічних операцій. Зокрема, операції розшифрування та створення цифрового підпису зазвичай потребують більше обчислень для довших ключів. Водночас збільшення довжини ключа підвищує стійкість RSA до атак, заснованих на факторизації модуля.

Самостійна реалізація генерації RSA-ключів у прикладному програмному забезпеченні зазвичай недоцільна, оскільки потребує коректної реалізації арифметики великих чисел, надійного джерела випадковості та перевірених методів тестування простоти. Помилки в цих компонентах можуть призвести до вразливості ключів. Тому доцільно використовувати перевірені криптографічні бібліотеки, які реалізують необхідні алгоритми та враховують вимоги безпеки.

5 Аналіз ситуацій порушення безпеки

5.1 Ситуація 1. Передбачувана генерація ключа

Проблема полягає в тому, що для створення AES-256 ключа використовується поточний час. Хоча SHA-256 повертає значення довжиною 256 біт, це не означає, що ключ має 256 біт криптографічної стійкості. Якщо зловмисник знає приблизний час створення ключа, він може перебрати можливі значення часу, обчислити їхні хеші та знайти потрібний ключ. Таким чином, безпека залежить не від довжини хешу, а від кількості можливих початкових значень. Для усунення проблеми необхідно генерувати ключ за допомогою криптографічно стійкого генератора випадкових чисел (CSPRNG), який забезпечує достатню непередбачуваність.

5.2 Ситуація 2. Однаковий seed на різних пристроях

Якщо однакові пристрої використовують однакове початкове значення для генератора псевдовипадкових чисел, вони можуть отримувати однакові послідовності. Якщо ці послідовності використовуються для створення криптографічних ключів, різні пристрої можуть мати однакові ключі, що значно знижує безпеку системи. У такому випадку компрометація одного пристрою може створити загрозу для інших пристроїв із такими самими ключами. Для розв'язання проблеми необхідно використовувати системний CSPRNG, який отримує випадкові дані з надійних джерел операційної системи, а не встановлювати однакове фіксоване початкове значення.

5.3 Ситуація 3. Розголошення правильно згенерованого ключа

Навіть якщо ключ було створено за допомогою надійного CSPRNG, систему не можна вважати повністю захищеною, якщо значення ключа записується до журналу програми. Журнали можуть бути доступними іншим користувачам або зберігатися тривалий час, тому зловмисник, який отримає до них доступ, зможе використати ключ для розшифрування даних або виконання інших операцій від імені системи. Для усунення проблеми не можна записувати секретні ключі до журналів. Якщо ключ уже було розголошено, його потрібно замінити, а доступ до журналів обмежити.

5.4 Ситуація 4. Недостатньо випадкова генерація RSA-ключа

Під час створення RSA-ключів джерело випадковості має критично недостатню ентропію, внаслідок чого різні ключі можуть бути сформовані з пов'язаних або повторно використаних простих чисел.

Основною проблемою тут є низька якість випадкового матеріалу, який використовується для генерації простих множників p та q. Якщо у двох незалежних модулів $N_1 = p_1 \cdot q_1$ та $N_2 = p_2 \cdot q_2$ випадково співпаде хоча б один простий множник (наприклад, $q_1 = q_2$), зловмисник зможе легко знайти цей спільний дільник за допомогою алгоритму обчислення найбільшого спільного дільника через формулу $\gcd(N_1, N_2) = q$. Це повністю компрометує та зламає обидві пари асиметричних ключів, дозволяючи розшифрувати приватний трафік або підробити цифрові підписи. Способом усунення цієї проблеми є використання виключно апаратно підкріплених системних криптографічно стійких генераторів (CSPRNG) із високим рівнем ентропії та попередньою перевіркою згенерованих чисел на унікальність [17, 13].

5.5 Ситуація 5. Надмірно тривале використання одного ключа

Інформаційна система використовує один (*master key*) для захисту даних усіх користувачів протягом тривалого часу без заміни.

Проблема полягає у відсутності політики ротації ключового матеріалу та порушенні принципів мінімізації ризиків. Така схема створює загрозу накопичення зашифрованого трафіку для подальшого криптоаналізу та критично збільшує масштаб наслідків у разі витоку ключа: у разі компрометації єдиного *master key* під загрозою опиняться абсолютно всі дані користувачів за весь період експлуатації системи. Роль регулярної ротації ключів полягає у зменшенні обсягу даних, захищених одним ключем, і локалізації наслідків можливої компрометації, оскільки зловмисник зможе отримати доступ лише до сегмента інформації, зашифрованого поточним зламаним ключем на конкретному часовому інтервалі [14].

6 Життєвий цикл криптографічного ключа

Спрощена схема життєвого циклу криптографічного ключа охоплює такі послідовні етапи: генерація → зберігання → використання → ротація/заміна → відкликання або знищення. Для забезпечення безпеки кожен із цих етапів вимагає врахування специфічних загроз та дотримання суворих вимог.

На етапі генерації ключа основною загрозою є використання передбачуваних алгоритмів або недостатньої ентропії (наприклад, застосування звичайних некриптографічних PRNG чи слабкого початкового значення *seed*), що дозволяє зловмиснику відтворити ключовий матеріал шляхом повного перебору. Вимогою безпеки тут є обов'язкове використання апаратно підкріплених криптографічно стійких генераторів випадкових чисел (CSPRNG) у захищеному середовищі [11].

На етапі зберігання ключа виникають загрози несанкціонованого доступу до файлової системи, конфігураційних файлів чи вихідного коду програми, де ключі можуть зберігатися у відкритому вигляді, а також ризики витоку через дампи пам'яті. Вимогою безпеки є розміщення ключів виключно в ізольованих шифрованих сховищах з використанням обмеження прав доступу та заборони експорту (*non-exportable keys*) [14].

Під час використання ключа основною загрозою є перехоплення секретних даних в оперативній пам'яті, атаки за побічними каналами (часові атаки або аналіз кешу процесора), а також випадковий запис ключа до системних журналів (логів) під час діагностики. Вимогами безпеки на цьому етапі є виконання криптографічних операцій із застосуванням алгоритмів із постійним часом виконання (*constant-time implementation*), ретельна фільтрація журналів та оперативне очищення ділянок пам'яті після завершення обчислень [14].

На етапі ротації або заміни ключа ключовою загрозою є відсутність регулярного оновлення ключового матеріалу, що призводить до надмірного накопичення зашифрованих даних під захистом одного ключа та збільшує масштаб можливої компрометації. Вимогою безпеки є впровадження автоматизованої політики планової ротації за розкладом або після обробки визначеного обсягу інформації з підтримкою контрольованого перехідного періоду [14].

На завершальному етапі відкликання або знищення основними загрозами є неповне або поверхневе видалення ключа, внаслідок якого залишки ключової інформації можуть бути відновлені зі стаціонарних носіїв чи кешу, а також затримка блокування компрометованого ключа системою. Вимогами безпеки тут є реалізація гарантованого безповоротного затирання пам'яті (процедури *zeroization* або *explicit_ bzero*) та миттєве внесення скомпрометованих ключів до чорних списків відкликання [15].

Висновки

Під час виконання лабораторної роботи ми дослідили теоретичні та практичні засади формування криптографічно стійких випадкових послідовностей, принципи генера-

ції ключового матеріалу та побудови асиметричних криптосистем. Аналізуючи сутність ентропії та різницю між генераторами псевдовипадкових чисел, ми з'ясували, що звичайні некриптографічні алгоритми моделювання та статистики є непридатними для захисту інформації через їхню передбачуваність, тоді як криптографічно стійкі генератори (*CSPRNG*) через системні API операційної системи забезпечують необхідний рівень непередбачуваності на основі збору апаратного фонового шуму.

Дослідження тестування простоти великих чисел показало, що використання високоефективних імовірнісних алгоритмів, зокрема тесту Міллера–Рабіна з достатньою кількістю ітерацій, дозволяє з мізерною ймовірністю помилки успішно генерувати прості множники, критичні для асиметричних криптосистем на зразок RSA. Практична частина роботи наочно продемонструвала відтворюваність звичайних *PRNG* порівняно з унікальними ключами *CSPRNG*, а також підтвердила, що час генерації RSA-ключів суттєво залежить від їхньої бітової довжини (наприклад, перехід від 2048 до 4096 біт) та складності обчислювальних процедур.

Аналіз порушень безпеки виявив, що навіть математично стійкі криптографічні алгоритми стають вразливими внаслідок використання слабких джерел ентропії, однакових початкових значень (*seed*) на пристроях, витоку секретів у системні журнали або відсутності регулярної ротації ключового матеріалу. З огляду на це, безпека інформаційної системи вимагає комплексного підходу на всіх етапах життєвого циклу ключа — від його апаратно підкріпленої генерації у захищеному середовищі до контрольованого відкликання та гарантованого знищення (*zeroization*) в оперативній пам'яті.

Перелік використаних джерел

Література

[1] Eastlake, D., Schiller, J., Crocker, S. *Randomness Requirements for Security*. RFC 4086, 2005.
`https://www.rfc-editor.org/rfc/rfc4086`

[2] Barker, E., Kelsey, J. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*. NIST Special Publication 800-90A Revision 1, 2015.
`https://csrc.nist.gov/pubs/sp/800/90/a/r1/final`

[3] *Recommendation for the Entropy Sources Used for Random Bit Generation*. NIST Special Publication 800-90B, 2018.
`https://csrc.nist.gov/pubs/sp/800/90/b/final`

[4] Python Software Foundation. *random — Generate pseudo-random numbers*. Python Documentation.
`https://docs.python.org/3/library/random.html`

[5] Python Software Foundation. *secrets — Generate secure random numbers for managing secrets*. Python Documentation.
https://docs.python.org/3/library/secrets.html

[6] Rivest, R. L., Shamir, A., Adleman, L. *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM, 1978.
https://doi.org/10.1145/359340.359342

[7] Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R., Simon, S. *Recommendation for Pair-Wise Key-Establishment Schemes Using Integer Factorization Cryptography*. NIST Special Publication 800-56B Revision 2, 2019.
https://csrc.nist.gov/pubs/sp/800/56/b/r2/final

[8] *Fermat's Little Theorem*. Wolfram MathWorld.
https://mathworld.wolfram.com/FermatsLittleTheorem.html

[9] Miller, G. L. *Riemann's Hypothesis and Tests for Primality*. Journal of Computer and System Sciences, 1976.
https://doi.org/10.1016/0022-0000(76)90043-8

[10] Rabin, M. O. *Probabilistic Algorithm for Testing Primality*. Journal of Number Theory, 1980.
https://doi.org/10.1016/0022-314X(80)90084-0

[11] *NIST SP 800-90A Rev. 1. Recommendation for Random Number Generation Using Deterministic Random Bit Generators*.
https://csrc.nist.gov/pubs/sp/800/90/a/r1/final

[12] *cryptography*.
https://cryptography.io/en/latest/

[13] *RFC 8017. PKCS #1: RSA Cryptography Specifications Version 2.2*.
https://www.rfc-editor.org/rfc/rfc8017

[14] *NIST SP 800-57 Part 1 Rev. 5. Recommendation for Key Management: General Guidance*.
https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final

[15] *FIPS PUB 140-3. Security Requirements for Cryptographic Modules*.
https://csrc.nist.gov/pubs/fips/140-3/final

[16] *Handbook of Applied Cryptography* / A. Menezes, P. van Oorschot, S. Vanstone. — CRC Press, 2001 (Chapter 4).
https://cacr.uwaterloo.ca/hac/

[17] *NIST SP 800-56B Rev. 2. Recommendation for Pair-Wise Key Establishment Schemes Using Integer Factorization Cryptography*.
https://csrc.nist.gov/pubs/sp/800/56/b/r2/final